

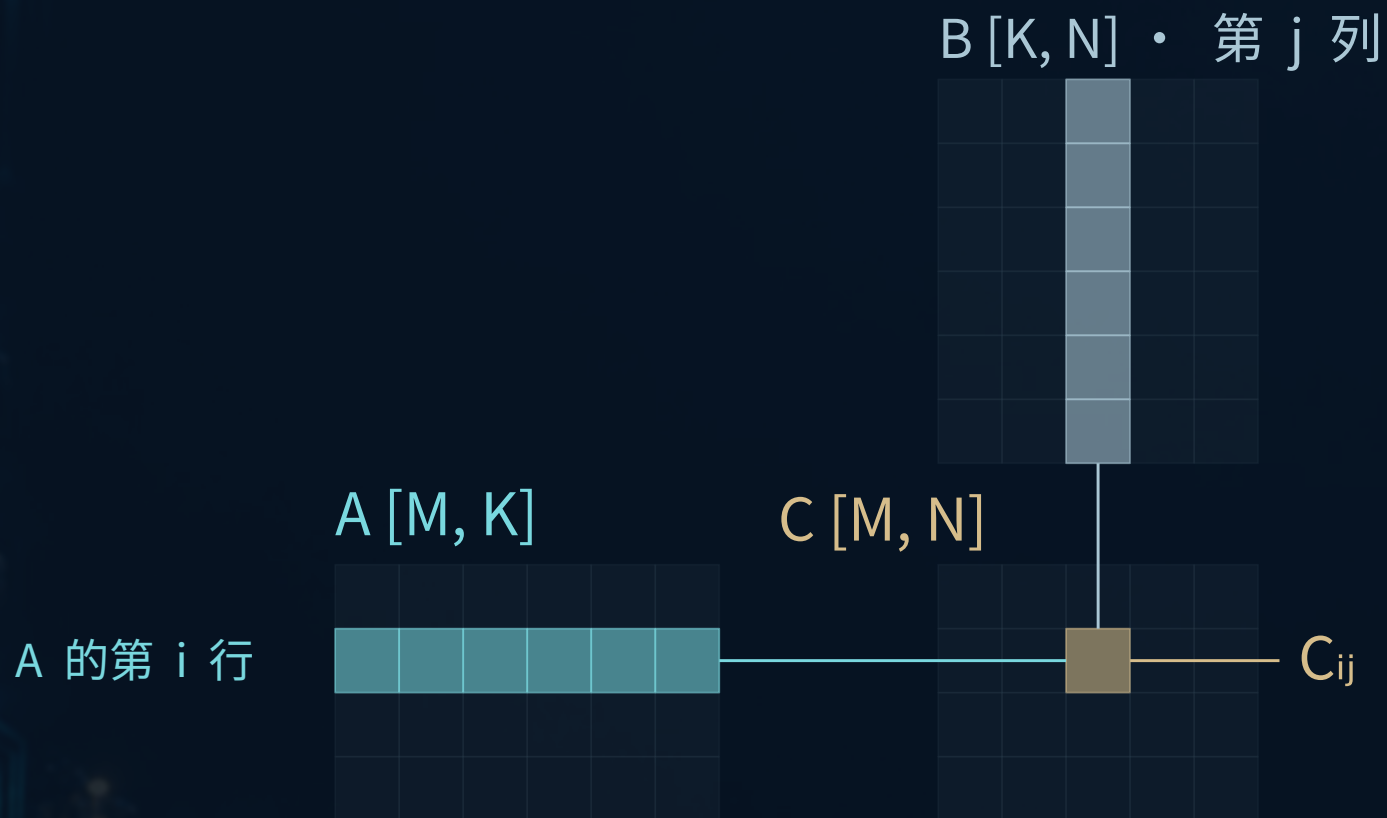
AI Infra: 智能背后的工程奇迹

从一块 GPU，到一个算力集群

在算力的物理边界，叩问 Scaling Law 的尽头

lny · github.com/HPC-SJTU/2026-freshman-ai-infra-lecture

基础数学：矩阵乘法与复杂度



矩阵乘法 $C = AB$
一行与一列做点积

$$C_{ij} = \sum_{k=1}^K A_{ik} B_{kj}$$

1 FLOP 表示一次浮点运算

(Floating-Point Operation)。

① 每个 C_{ij} 从 0 开始，累加 K 个乘积

② K 次乘法 + K 次加法 = $2K$ FLOPs

③ C 共 MN 个元素 $\rightarrow$ 总量 $2MNK$ FLOPs

大 O · 增长上界

对非负函数 f, g :

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, n_0, \forall n \geq n_0: f(n) \leq c \cdot g(n)$$

当规模足够大， f 的增长被 g 的常数倍所界定。

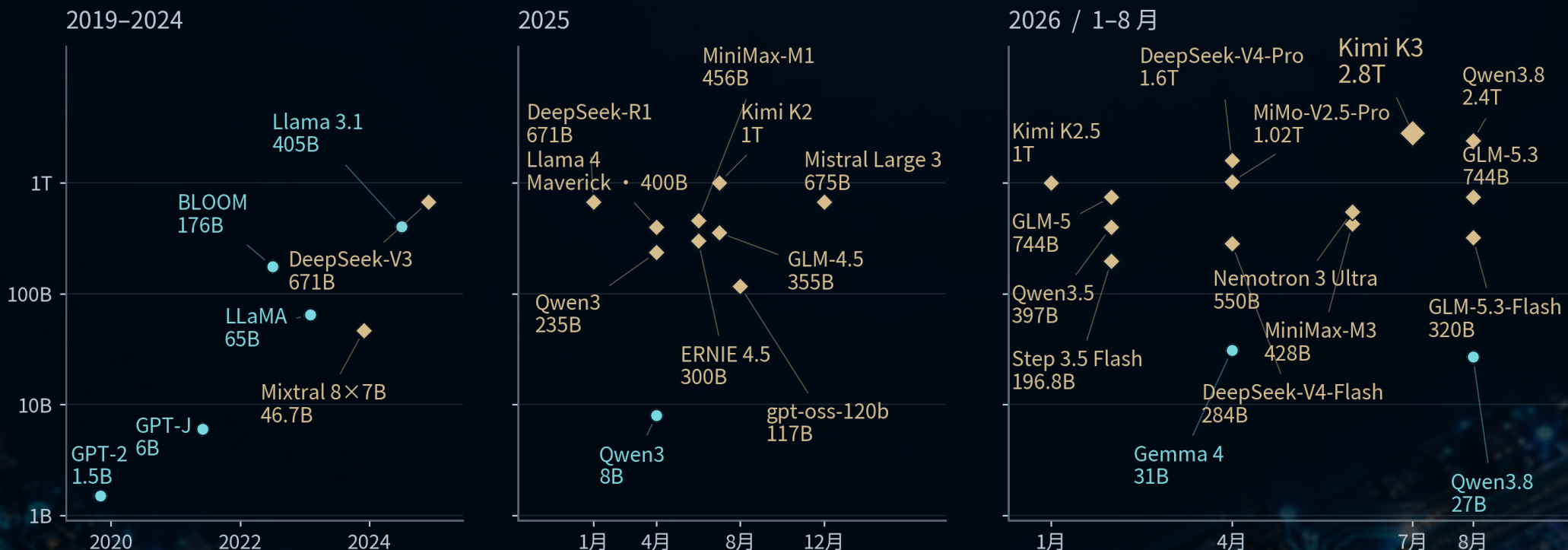
开放权重模型的规模扩展

32 个主流模型与关键版本 · 资料截至 2026-09-06

总参数量（对数刻度）

● Dense（稠密）

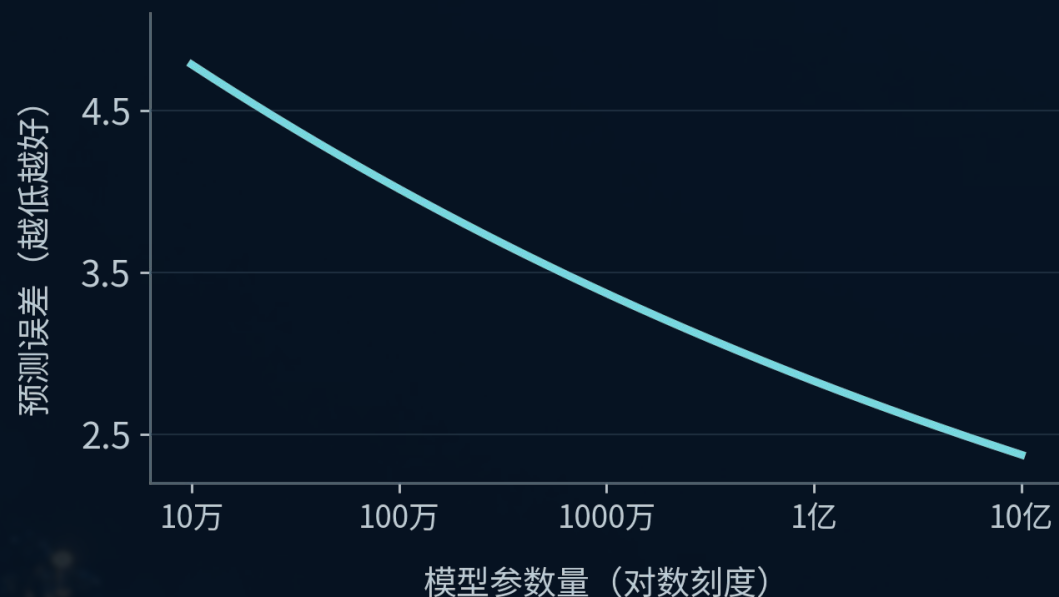
◆ MoE（混合专家）



来源：官方模型卡与发布公告，采用官方公布的参数规模。B = 十亿，T = 万亿；横轴按发布月份分段展示。

Scaling Laws

在数据质量、训练方法与计算预算配合时，规模扩展带来能力提升。



更大模型 + 更多数据



模型能力的可达上界 ↑

如何获得更多有效算力，
支撑进一步 Scaling？

来源：Kaplan et al. (2020)，式 (1.1) 拟合关系。

买更好的 GPU，够吗？

Kimi K3 · 官方 MXFP4 权重

约 1.56 TB

NVIDIA Rubin · 单 GPU

288 GB



MXFP4 文件总量



HBM4 容量

$\approx 5.4\times$

完整权重需要更大的 GPU 显存总容量

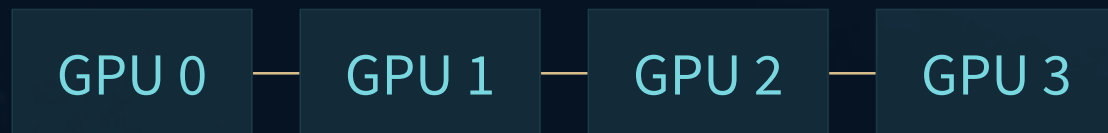
十进制 TB / GB · 运行时还需 KV、激活与工作区

来源：Kimi K3 官方模型仓库 · NVIDIA Vera Rubin 规格表

如何用多卡扩展容量？

加卡之后，如何变成有效算力？

协同组织，发挥多卡算力



高速互联 + 并行切分 / 协同 / 负载分配

扩容也受供货、预算与供电约束

AI Infra：把大规模集群组织成高效可用的算力，
支撑模型训练与推理。

同一 A100，优化 Attention 实现

标准 PyTorch → FlashAttention-2

17 → 140 TFLOP/s

≈ 8×

A100 80GB SXM4 · BF16

causal · FWD + BWD

来源：FlashAttention-2 · Fig. 4(c)

AI Infra 技术栈

Xflops 超算队
Supercomputer Competition Team

NVIDIA 计算生态

● 微观切入 ● 宏观切入

强化学习框架



slime



verl



OpenRLHF



AReal

训练框架



Megatron-LM



DeepSpeed



torch titan

推理框架



vLLM



TensorRT-LLM



SGLang



llama.cpp

张量框架



PyTorch



JAX

专用算子与通信库

FlashAttention



FlashInfer



DeepGEMM



DeepEP

图编译器



TorchInductor



XLA

算子语言与 DSL



CUDA C++



CuTe DSL



Triton



TileLang

基础算子与通信库



cuBLAS



cuDNN



CUTLASS



NCCL



NVSHMEM

内核编译器



Triton Compiler



nvcc

系统与计算平台



CUDA



Linux



NVIDIA Driver



OFED

计算与高速互联



NVIDIA GPU



NVLink



NVSwitch



Mellanox (IB)

目录

Xflops 超算队
Supercomputer Competition Team

Part 1 微观视角

局部计算的高效实现

Part 2 宏观视角

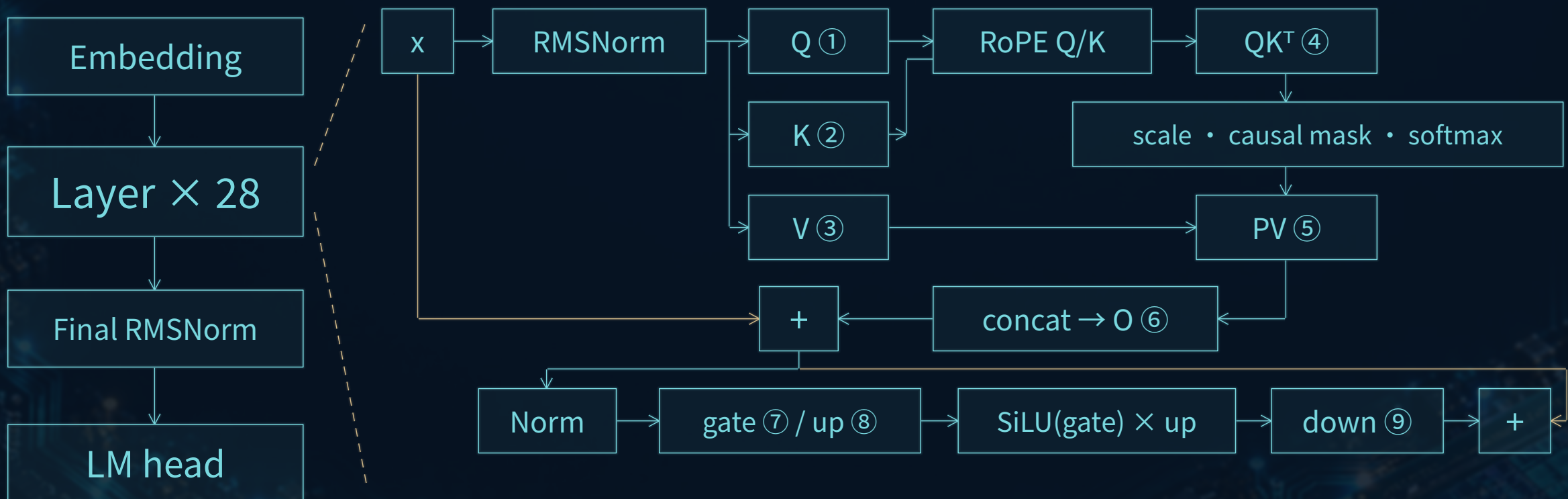
状态管理与多卡协同

Part 3 模型侧优化

紧凑表示与高效生成

反复出现的矩阵乘

Qwen2.5-7B：一个 layer，7 个投影 + 2 个注意力矩阵乘。



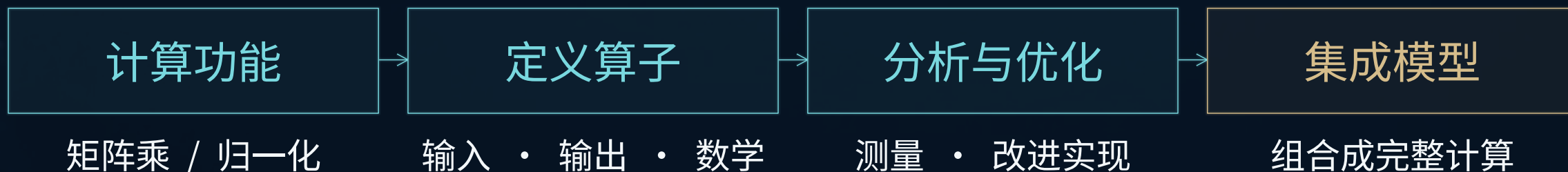
$V = 152064$

Q 投影示例: $[2048, 3584] @ [3584, 3584]$

金色实线表示残差；因果注意力读取当前 token 允许访问的全历史。

算子抽象

把反复出现的计算功能，整理成可独立分析与优化的单元。



缩小分析范围

用 *profile* 确定耗时热点

简化只是第一步，算子的优化才是核心的难点

矩阵乘性能优化

行主序 FP32 : $A[M,K] \times B[K,N] \rightarrow C[M,N]$

朴素 C++ 三循环

```
for (int i=0; i<M; ++i)
  for (int j=0; j<N; ++j) {
    float sum = 0;
    for (int k=0; k<K; ++k)
      sum += A[i*K+k]*B[k*N+j];
    C[i*N+j] = sum;
  }
```

1024^2 : 4.20 s $\rightarrow$ 25.9 ms

OpenBLAS · 专用微内核

Haswell SGEMM 微内核

424 行代码

分块、向量化、数据打包与调度

约 160x

把常用算子优化好能否直接搭积木实现高性能？

计算强度：计算与搬运

每搬运一字节，完成多少计算

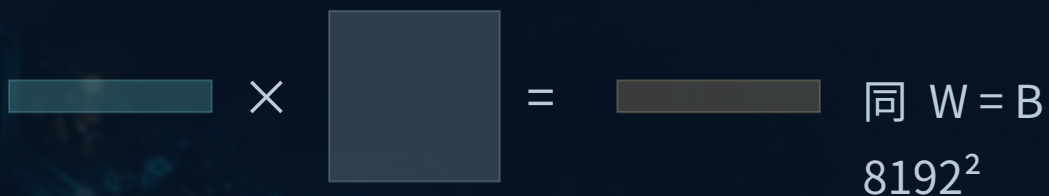
$$I = F / Q \cdot \text{单位: FLOP/Byte}$$

计算量 F

$$F = 2MNK \text{ FLOPs}$$

M 、 N ：输出行 / 列数； K ：归约维

$$\text{GEMV} \cdot M = 1, K = N = 8192$$



$$I = 2KN / [2(K + KN + N)]$$

$$\approx 1 \text{ FLOP/Byte}$$

同一份权重供更多行复用，单位搬运量支撑的计算就更多。

矩阵乘法 $C = AB$

$$A[M, K] \cdot B[K, N] \cdot C[M, N]$$

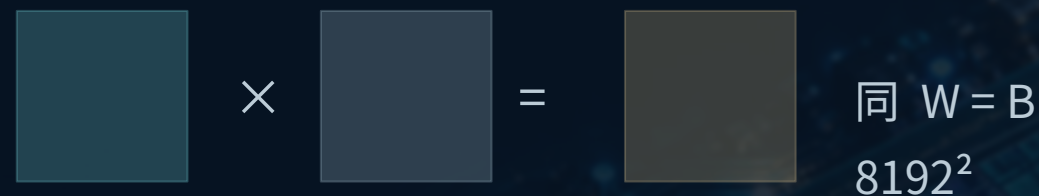
BF16 输入 / 输出：2 Byte / 元素

HBM：输入各读 1 次，新输出写 1 次

$$Q_{\text{理论}} = 2MK \text{ (读 } A) + 2KN \text{ (读 } B) + 2MN \text{ (写 } C)$$

$$= 2(MK + KN + MN) \text{ Byte}$$

$$\text{GEMM} \cdot M = N = K = 8192$$



$$I = 2 \cdot 8192^3 / (2 \cdot 3 \cdot 8192^2)$$

$$= 8192 / 3 \approx 2731 \text{ FLOP/Byte}$$

H200 Roofline : 强度与性能上界

H200 SXM • 稠密 BF16 ≈ 989 TFLOP/s • HBM 4.8 TB/s

理想性能上界 (TFLOP/s, log)



H200 SXM • cuBLAS 实测: 796 FLOP/Byte

轴上的点表示计算强度; Roofline 曲线表示对应的理想性能上界。

$I_{\text{理论}} = 2MNK / Q$ 输入各读一次输出写一次

$I_{\text{实际}} = 2MNK / Q_{\text{HBM}}$ 实际读写

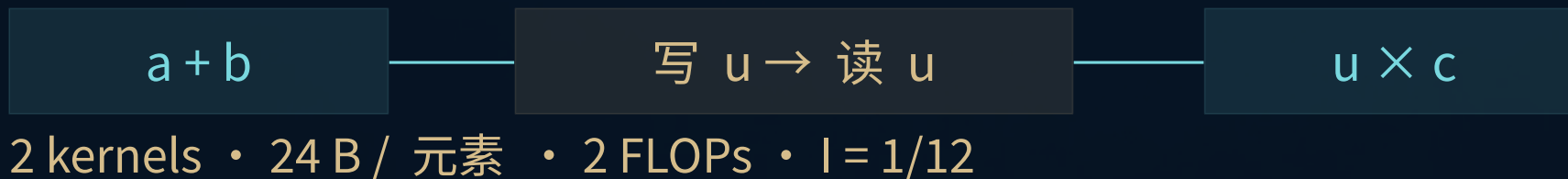
同一问题的有效计算量相同, 实际 HBM 流量由分块方式、片上容量与缓存复用共同决定。

算子融合

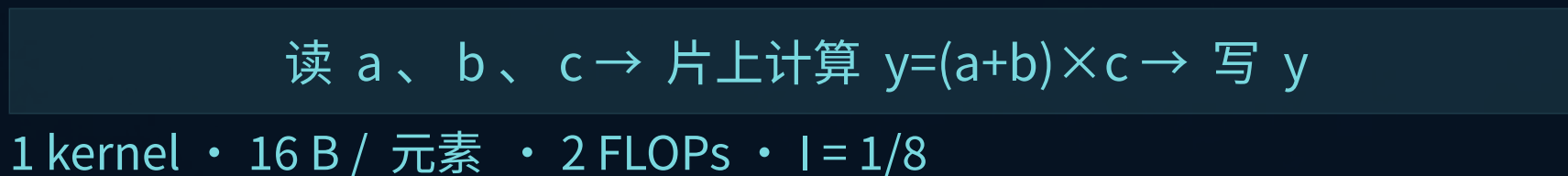
主收益：同样的有效计算 F ，减少搬运 Q ，提高实现计算强度

$u = a + b$; $y = u \times c$ · FP32 冷流式示例

分开



融合



- ① 避免中间写出再读 ② 复用已加载数据
- ③ 避免某些独立重排 / 转换的额外访存

补充收益

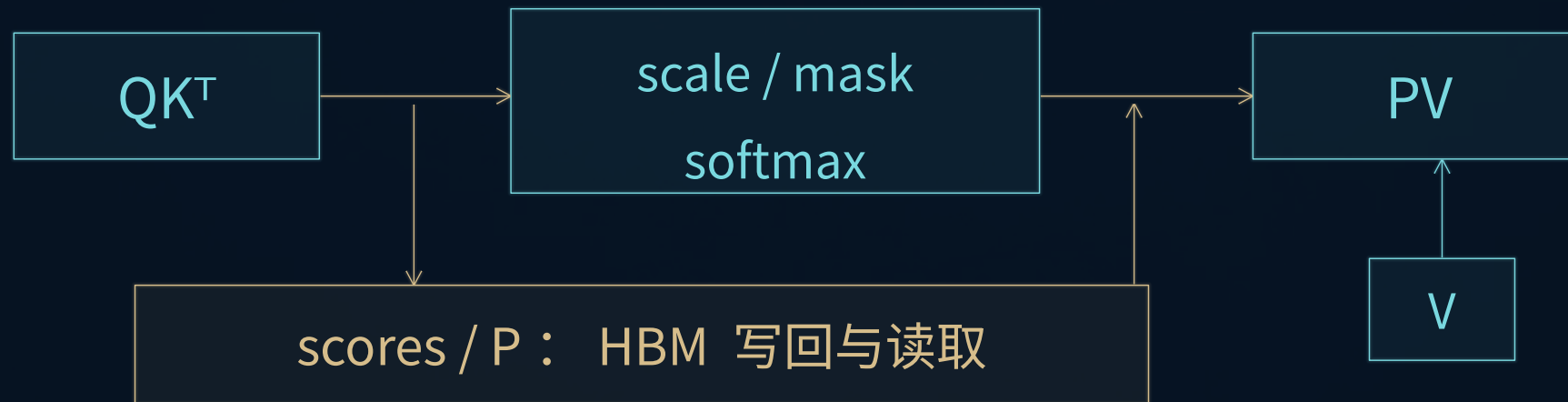
减少 launch / 调度 / 边界
改善流水 / 资源利用

消除冗余计算 / 指令
减少中间 buffer

FlashAttention

分块 $Q/K/V$ 、在线 softmax 与输出累积，把中间计算留在片上。

分步组织



分块融合

$Q/K/V$ 块 $\rightarrow$ 在线 softmax $\rightarrow$ 累积输出

如何实现高性能算子？

CUDA: 更多控制, 更高复杂度

开发者安排硬件执行细节, 用测试与测量驱动迭代。



执行映射

线程与并行数据布局

存储层次与同步

实现与验证

局部计算图的融合选择

编写、测试、测量与迭代

CUDA 开发负担重, 许多底层实现工作反复出现。

更高层的优化

用更高层的表达与自动化，把硬件优化变成可迭代的过程。

DSL 表达

Triton / CuTe DSL

TileLang

分块 · 布局 · 并行

图编译优化

torch.compile

图捕获 · 算子融合

实现选择 · 代码生成

Agent 驱动优化

KernelAgent

写代码 · 测试 · profile

迭代 CUDA / DSL / 图

更多算子开发方向

通算融合：跨 GPU 通信与计算的细粒度协同

Megakernel：多算子融合与 GPU 内任务调度

从局部计算的高效实现，转向状态与设备的整体协作。

KV Cache 复用

历史状态持续维护

多级存储按时机送达

多卡并行与通信

组织设备分工

协调数据交换与执行依赖

超级计算机通信究竟有多快？

典型多 GPU 超算集群由多个计算节点组成，每个节点配置多张 GPU，通过高速互联协同发挥算力。

10GbE · 每方向 / 链路

TB5 常规 · 每方向

DDR5 双通道 · 内存聚合

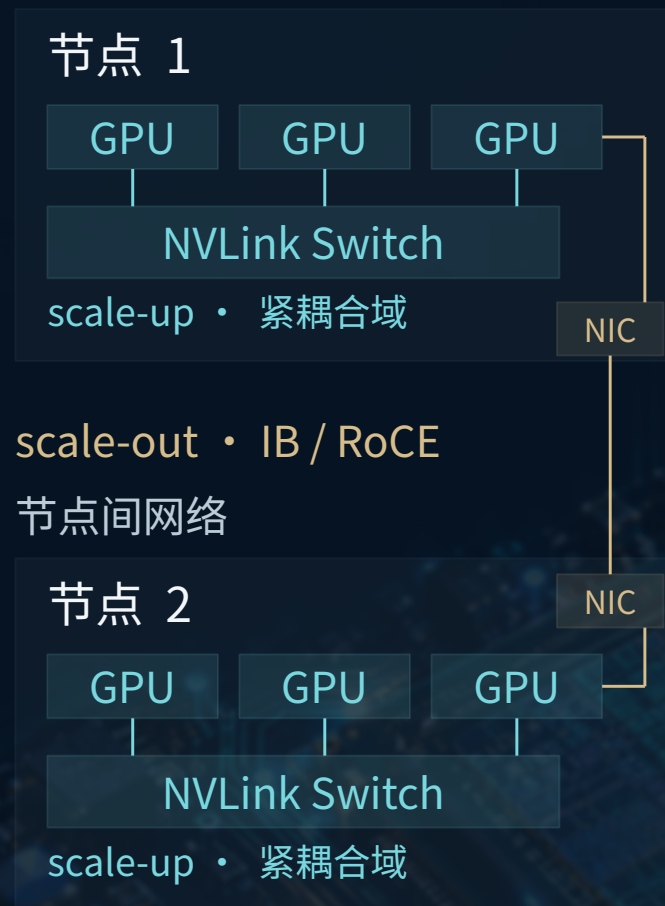
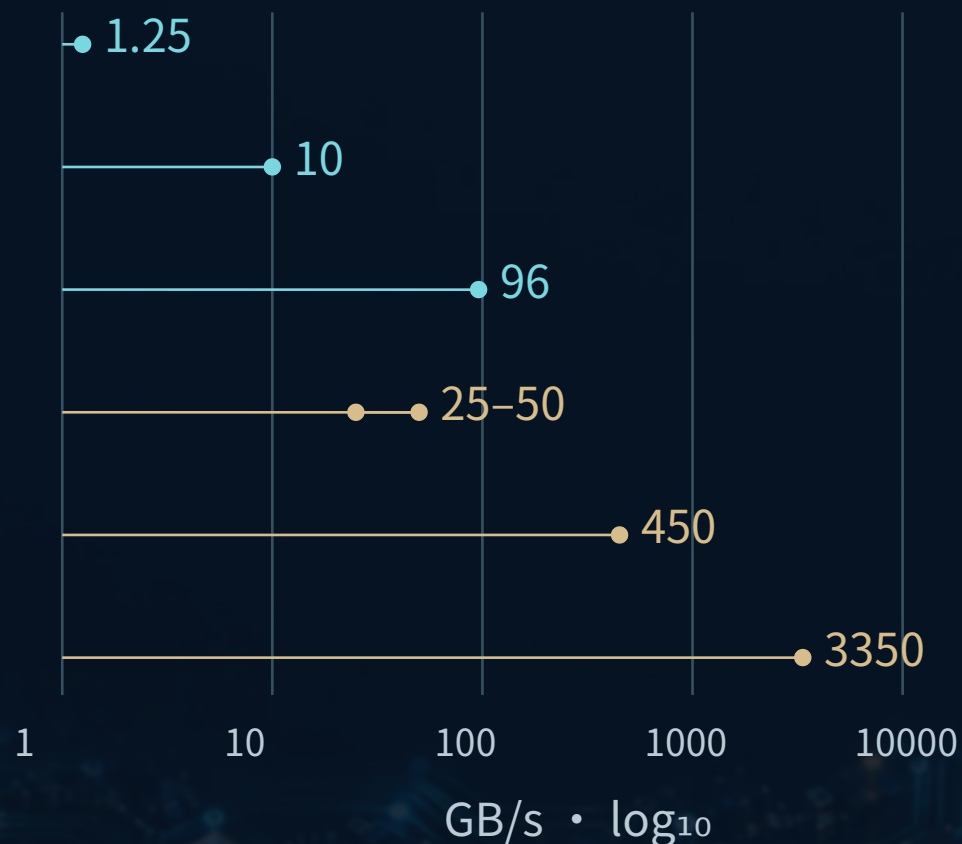
RDMA · 每方向 / 链路

H100 NVLink · 单向 / GPU

H100 HBM · 整 GPU

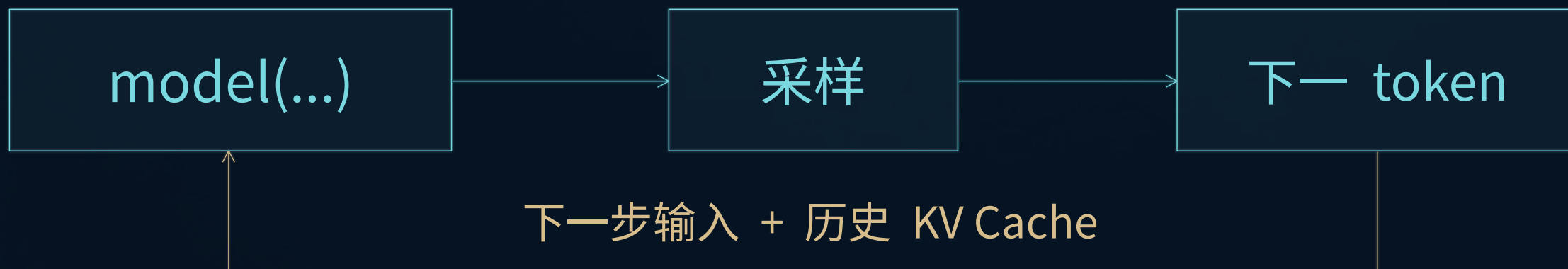
PC 参考

集群参考



十进制标称带宽；计量范围见各项标签。

模型已有 forward，推理框架还承担哪些工作？



生成持续维护历史状态；推理框架统一管理 KV Cache 并协调请求的计算资源与状态生命周期。

核心的状态：KV Cache

KV Cache 复用

各层 KV 按 token 位置示意 · 每轮处理一个新输入，复用历史缓存

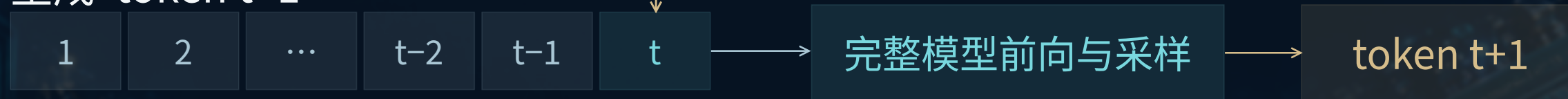
生成 token t



输入 t-1：生成 $Q[t-1]$ ，追加 $KV(t-1)$

下一步输入

生成 token t+1



输入 t：生成 $Q[t]$ ，追加 $KV(t)$

复用已有 KV

追加本轮 KV

上下文很长时，KV Cache 会有多大？

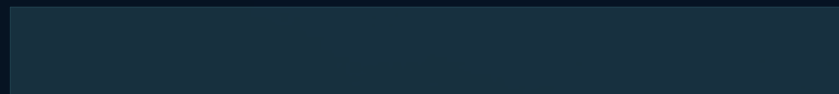
KV Cache 容量

GLM-5.3 • 744B 总参数 • 单请求 1M 上下文

87.75 GiB $\approx$ 94.22 GB

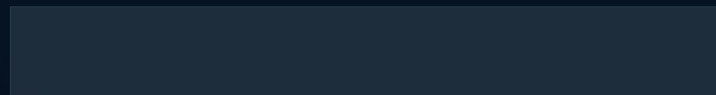
BF16 压缩 MLA 核心缓存 • 1M = 1,048,576 token • 按官方配置估算

GLM-5.3 核心 KV



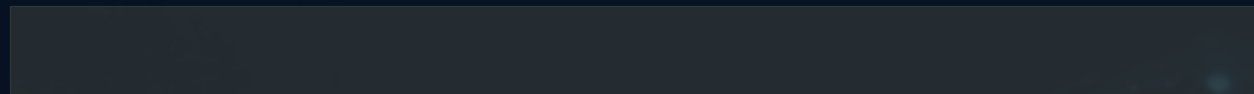
94.22 GB

H100 SXM 显存



80 GB

H200 SXM 显存



141 GB

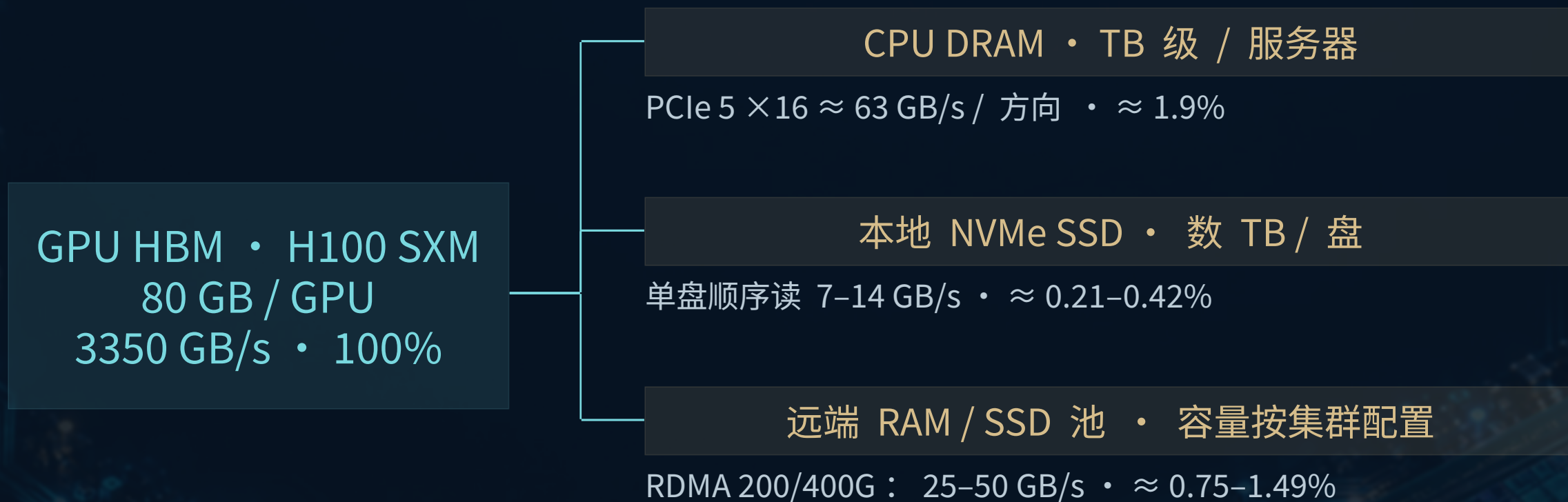
1M 上下文的 BF16 压缩 MLA 核心缓存约 88 GiB。

部署预算还包括权重、DSA 辅助状态、MTP 与运行工作区。

KV 需求超过显存预算时，如何组织存储？

多级缓存

从各存储位置按访问时机向 GPU 预取 / 加载



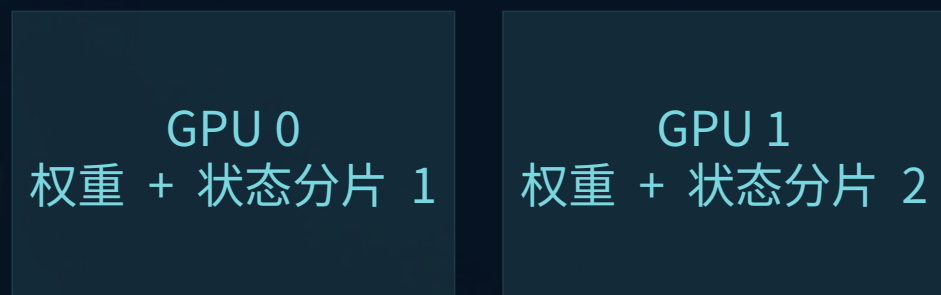
RDMA 高速取回 • 按平台支持选择 GPUDirect 或主机中转

LMCache • 存储 / 预取 / 加载；图示带宽为各传输路径参考规格

并行目标

先分清容量与性能，再选择设备组织方式。

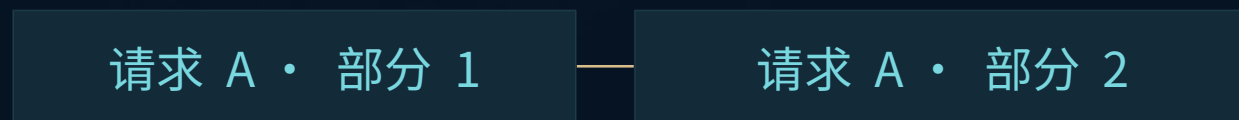
容量 · 装得下



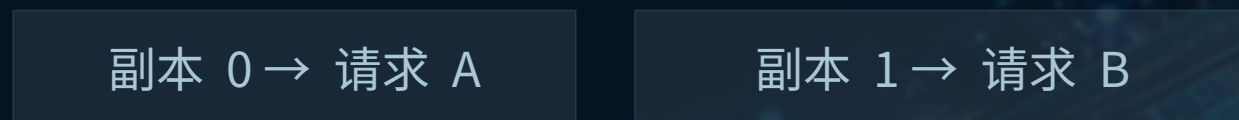
权重与状态分存

性能

时延 · 同一请求协作



吞吐 · 不同副本服务不同请求



性能收益有条件：看通信、负载与关键路径。

并行代价

AllReduce 需要双方结果；后续计算依赖归约完成。



通信时间 $\approx$ 次数 $\times$ 延迟 + bytes / 有效带宽

数据并行 DP

复制完整模型，让更多请求同时得到服务。



提升吞吐与并发，让就绪请求更快获得服务。

请求到达时分发输入、生成时返回输出，载荷随输入 / 输出 token 数增长。

完整副本适合 scale-out 部署，按请求负载配置副本数。

流水线并行 PP

按层分工，让不同请求在流水线中同时前进。

等长阶段示意；采样与回送开销合并到阶段边界。

t2 同时执行
C1 / B1 / A1

Rank 0 · 前段层

Rank 1 · 中段层

Rank 2 · 后段层

边界激活 [T,H]
Send / Recv



按层分存权重 / KV，利用多请求流水提升吞吐。

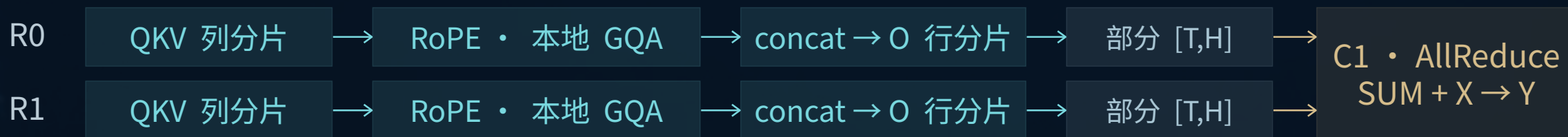
每 microbatch / 生成步跨 stage 传激活，单次逻辑载荷 $T \times H \times b$ 。

可跨节点 scale-out，结合阶段平衡、就绪请求与边界传输组织流水。

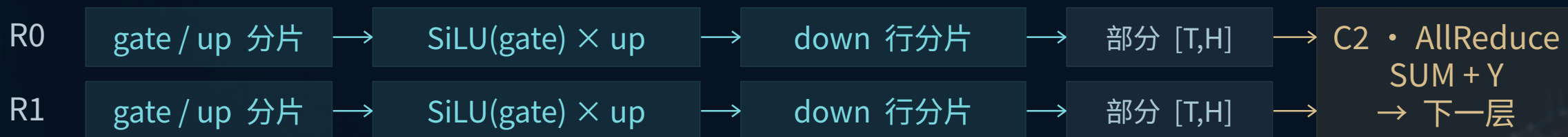
张量并行 TP

Qwen2.5-7B · TP = 2 · 同层计算分片，归约连接下一步

$X \rightarrow \text{RMSNorm} \rightarrow \text{Attention}$



$Y \rightarrow \text{RMSNorm} \rightarrow \text{SwiGLU MLP}$



每次归约 $[T, 3584] \cdot \text{BF16} \cdot T=2048 \rightarrow 14 \text{ MiB}$

step k : $L1 [C1 \ C2] \rightarrow \dots \rightarrow L28 [C1 \ C2] \cup \text{step } k+1$

经典 TP 前向：每层 2 次 AllReduce，28 层共 56 次，随生成步持续重复。

分摊模型容量，争取更低时延与更高单实例吞吐。

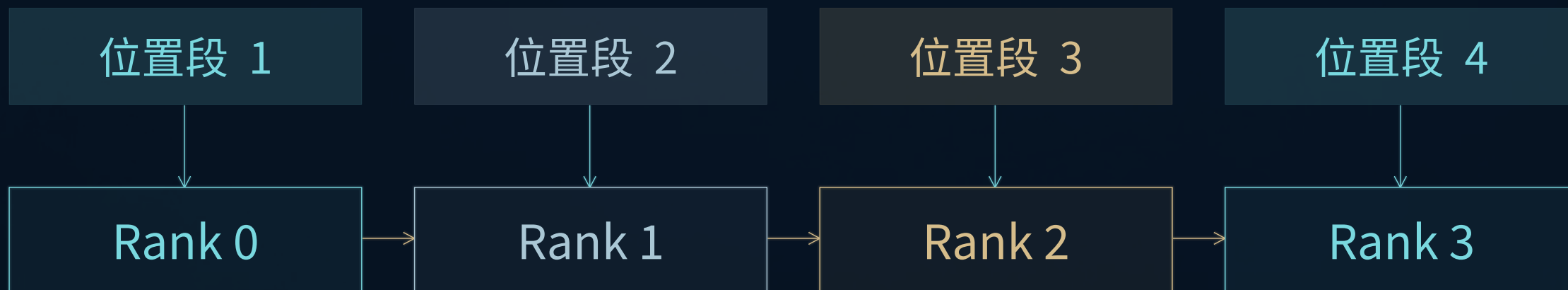
激活 / 部分结果随每层、每步归约，单次载荷 $T \times H \times b$ 。

优先 scale-up；跨节点效率与固定总卡数的副本配置一起评估。

上下文并行

CP：把同一条长序列的位置区间分给多张 GPU。

同一请求的长序列



扩展长上下文 KV / 激活容量，争取更优 TTFT / TPOT。

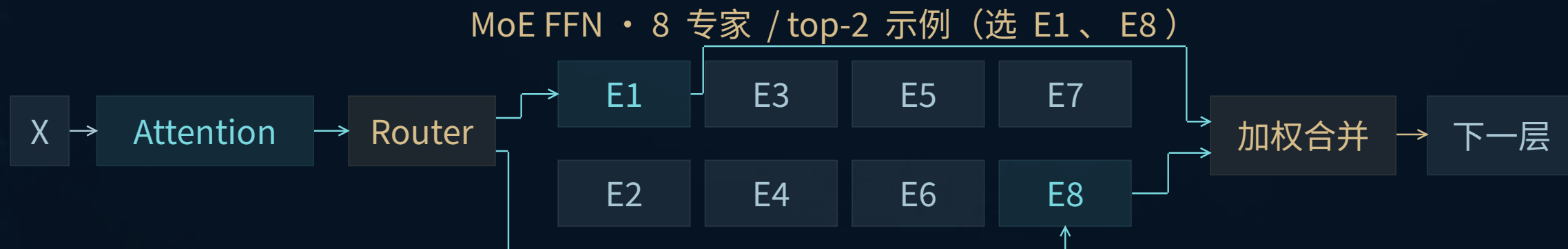
Prefill：长序列提供充足计算，通信可与计算重叠。

Decode：每步计算较短，通信掩盖空间有限，更容易进入关键路径。

互联选择：Prefill 可考虑 scale-out 域；Decode 优先 scale-up 域。

MoE 模型与专家并行 EP

模型结构：Attention 处理交互，MoE FFN 按路由计算



一个 EP 组 · 4 GPU / 完整 8 专家

GPU 0		GPU 1		GPU 2		GPU 3	
E1	E2	E3	E4	E5	E6	E7	E8

分摊专家权重容量，争取更低时延与更高吞吐。

每 MoE 层 dispatch / combine 隐藏向量，载荷随 $\text{top-k} \times \text{token}$ 增长。

scale-up / scale-out 均常见，结合网络与负载均衡组织专家。

MoE 混合并行：Attention 与专家分阶段布局

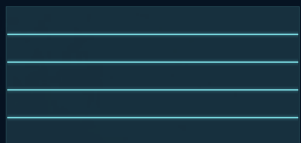


同一组 Rank 0-3：Attention 可选一种划分策略；MoE 按专家划分，输出继续进入下一层。

推理负载

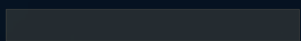
输入与输出阶段，给同一模型带来不同的计算形状。

Prefill



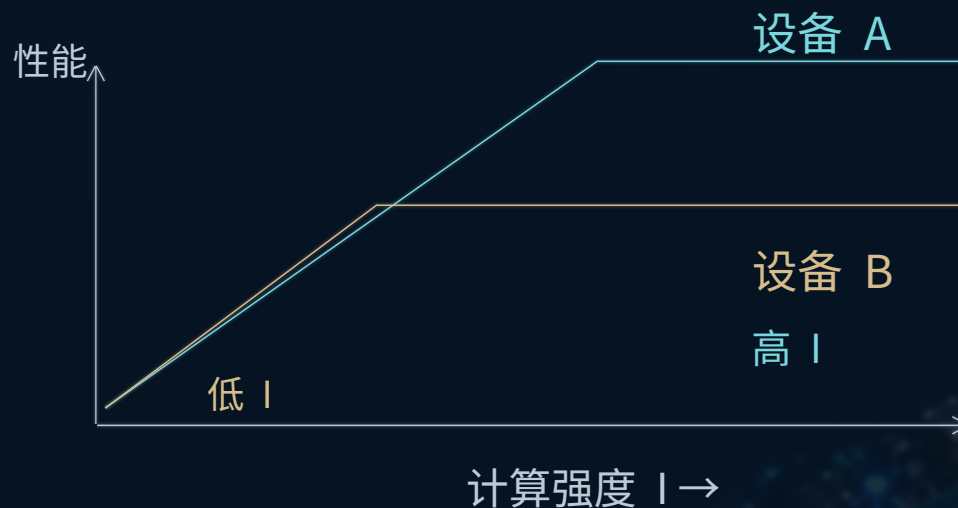
许多 token 行 $\rightarrow$ 大 GEMM
高 I · 通常更偏算力

Decode · 小 batch



单请求 GEMV / 小批窄 GEMM
低 I · 通常更偏带宽

两种设备的 Roofline 示意



批处理 $\times$ 并行方式 $\times$ 实例数量，按阶段独立调优

结合带宽、算力、HBM 容量、网络、成本与服务 SLO 匹配设备。

PD 分离

提高满足 TTFT / ITL 约束的有效服务吞吐（SLO goodput）。



稳定流式输出

隔离长 Prefill 与持续 Decode
减少干扰，控制尾部 ITL

分阶段独立调优

batch / TP / PP / 调度
分别优化 TTFT 与 ITL

按负载独立扩缩容

按到达率和输入 / 输出长度
调整 P / D 实例数量

同型号 GPU 分池，独立组织 P / D 资源以提升 SLO goodput。

每请求交接 KV + 生成状态，可分块传输，载荷随上下文增长。

scale-out RDMA；结合负载、KV 交接与调度评估收益。

理解系统瓶颈，就能解释模型优化。

容量与带宽

更紧凑的表示

权重精度 · KV 宽度

历史计算与读取

更高效的注意力

选位置 · 压序列 · 状态

数据复用

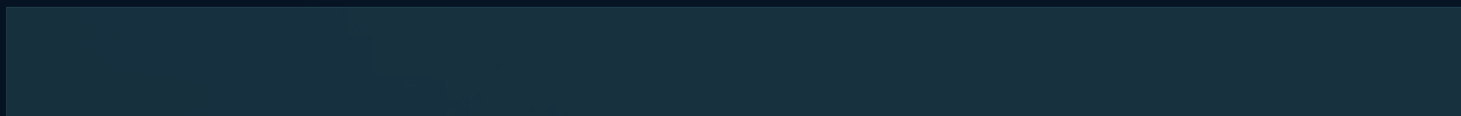
多 Token 并行验证

一次读权重，多做计算

低精度权重

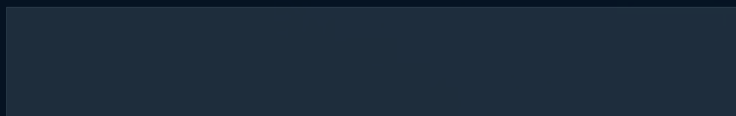
较少可表示值配合 scale 等参数，近似表示模型权重。

16 bit



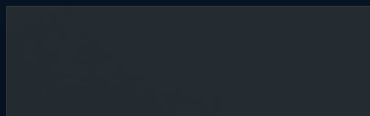
140 GB

8 bit



70 GB

4 bit



35 GB

70B 模型 • 纯权重 payload : $1 \rightarrow 1/2 \rightarrow 1/4$

更小权重节省容量与带宽；配套 kernel 发挥紧凑表示的性能收益。

gpt-oss : MoE 权重采用 MXFP4 ; 其他张量、scale 与 KV 独立预算。

Multi-Head Latent Attention

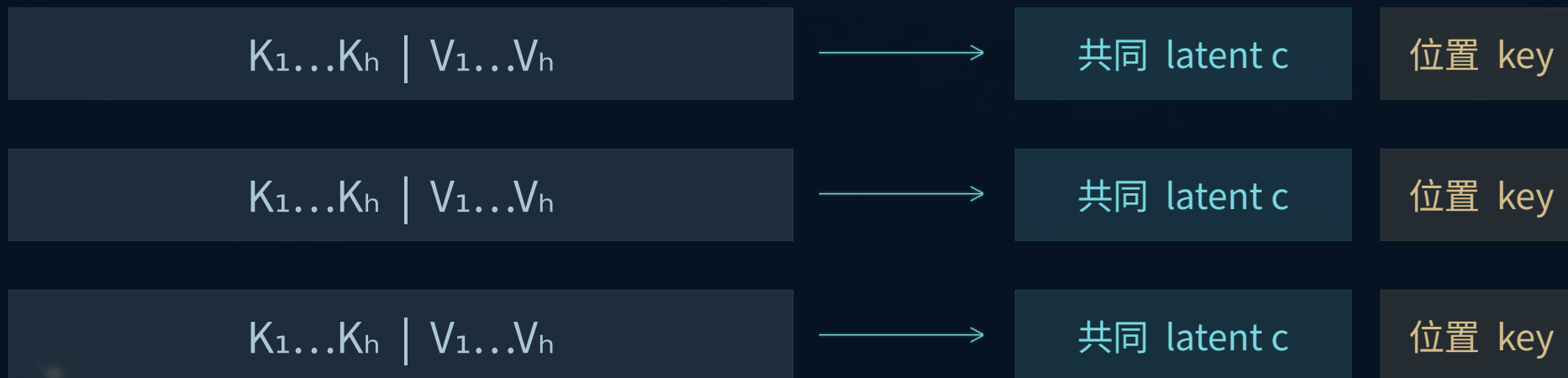
多头潜在注意力 · MLA

DeepSeek-V2 / V3 · MLA : 让每个历史 token 的记录更窄。

逐 token 的历史记录

$c = W_{\text{down}} h$

共享 RoPE key



完整多头 K / V

学习到的窄表示 + 独立位置项

查询侧吸收 K 上投影 · 聚合 latent 后再映射输出

每条记录更窄；历史条目数随 N 增长，稠密 MLA 每步读取 $O(N)$ 历史。

DeepSeek Sparse Attention

稀疏注意力 · DSA

每步注意力计算能否更少？

DeepSeek-V3.2 · DSA : 轻量筛选 → 主注意力



主注意力: $O(N) \rightarrow O(k)$

$k \leq \min(2048, N)$

索引扫描与历史缓存为 $O(N)$; 主注意力读取选中的 k 个条目, 完整历史供后续查询选择。

Heavily Compressed Attention

高度压缩注意力 · HCA

如何减少需要保存的历史 KV 条目？



DeepSeek-V4 Preview · HCA：读取全部压缩历史与近期窗口

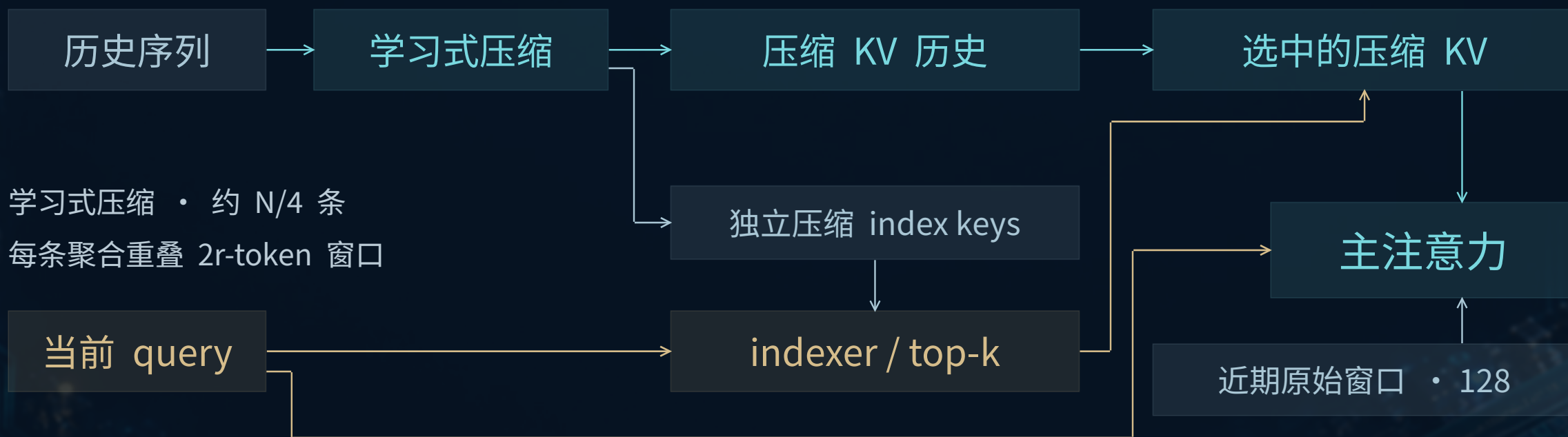
$O(N/r + w)$ · $r = 128$ · $w = 128$ ；摘要数量仍随历史增长。

压缩后的历史，是否还需要全部读取？

Compressed Sparse Attention

压缩稀疏注意力 · CSA

DeepSeek-V4 Preview · 压缩后筛选，直接读取选中的压缩 KV



主注意力 $O(k+w)$ ；索引 $O(N/r) + \text{selection}$ ；缓存随 N/r 增长

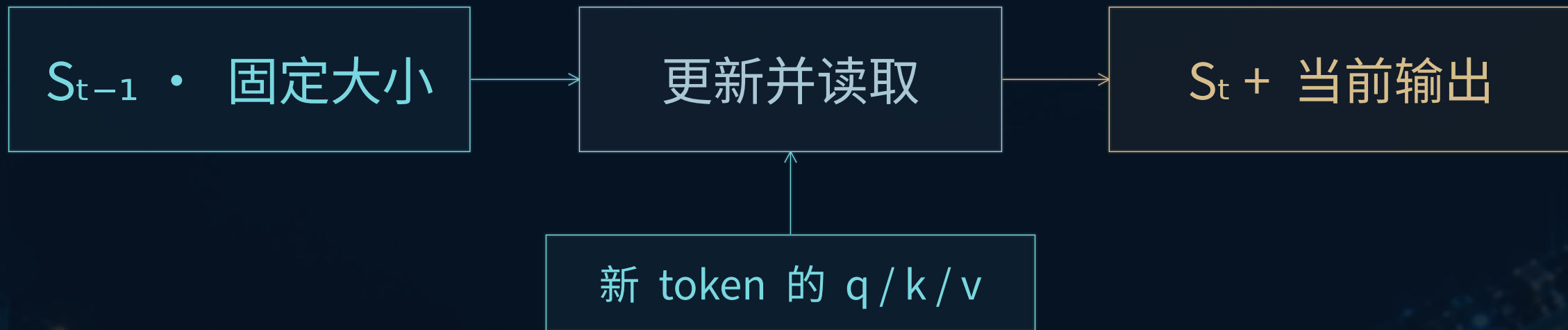
最新进展 · CSA2：跨层复用 KV 与稀疏选择结果，进一步减少重复存储和索引。

即使压缩并筛选，历史缓存仍会增长；能否改用固定大小的状态？

Linear Attention

线性注意力

能否用固定大小的状态，持续更新历史信息？



递归层：每步 $O(1)$ · 历史状态 $O(1)$

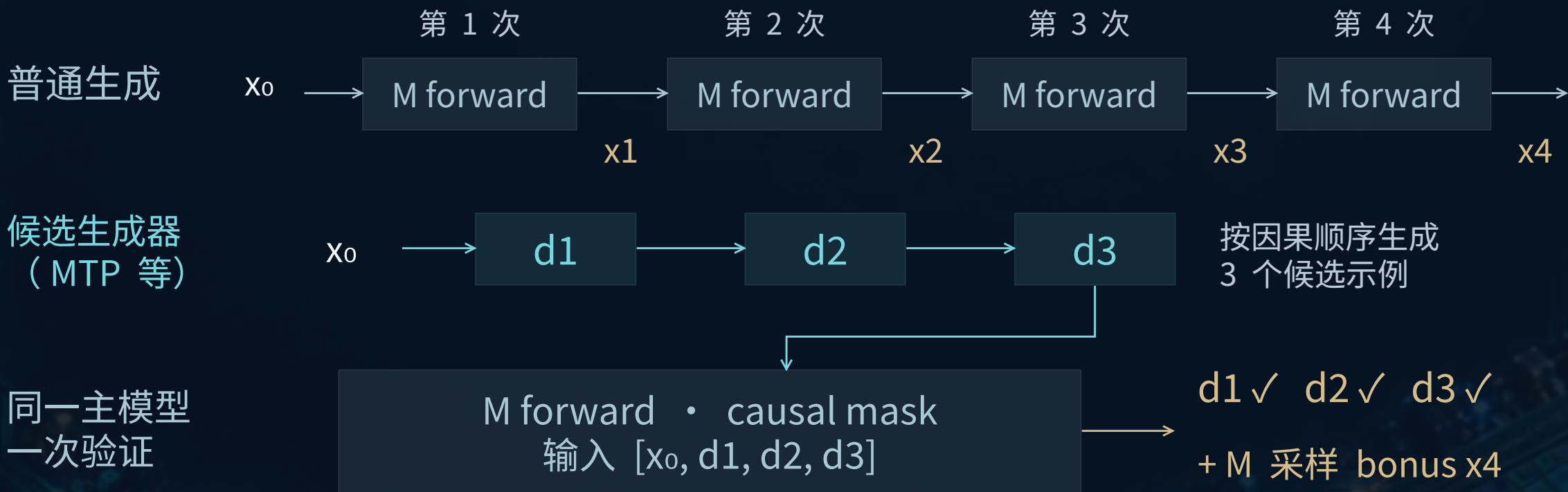
Gated DeltaNet / Qwen3-Next · KDA / Kimi Linear

固定模型维度、相对 N：递归层维护固定状态；Full / MLA 层按历史长度扩展，状态设计兼顾表示质量。

MTP：多 Token 预测

Multi-Token Prediction

共同起点：Prefill 与所需状态就绪； x_0 已采样，尚未送入下一次 M forward。



全接受示例：3 个候选 + 1 个主模型采样 token (x_0 不计入)

每 token 成本 = 每轮总时间 / 平均有效 token 数

共同基础：Linux · Python · PyTorch

单卡性能

C/C++ · 组成与存储
并行计算 · 性能分析
入口：CUDA / Triton

多卡与集群

操作系统 · RDMA 网络
集合通信 · 分布式调度
入口：NCCL / vLLM

推荐教程

北京大学未名超算队

AI Infra 系列讲座

哔哩哔哩 · 扫码观看



计算资源

数据中心 GPU · 单机多卡

多机高速网络互联环境

实战项目

算子优化 · 推理服务调优

并行策略设计 · 通算重叠优化

交流平台

AI Infra 快速变革，持续追踪前沿

技术交流 · 经验分享 · 共同探索

学习资源

计算资源

数据中心 GPU · 单机多卡
多机高速网络互联环境

实战项目

算子优化 · 推理服务调优
并行策略设计 · 通算重叠优化

交流平台

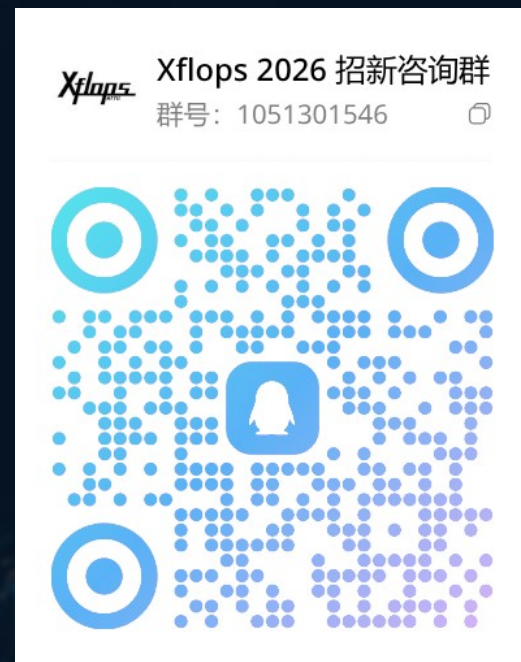
AI Infra 快速变革，持续追踪前沿
技术交流 · 经验分享 · 共同探索

校超算队，为大家提供

学习与实践的平台。

欢迎加入校超算队

扫码加入招新咨询群



学习资源

计算资源

数据中心 GPU · 单机多卡
多机高速网络互联环境

实战项目

算子优化 · 推理服务调优
并行策略设计 · 通算重叠优化

交流平台

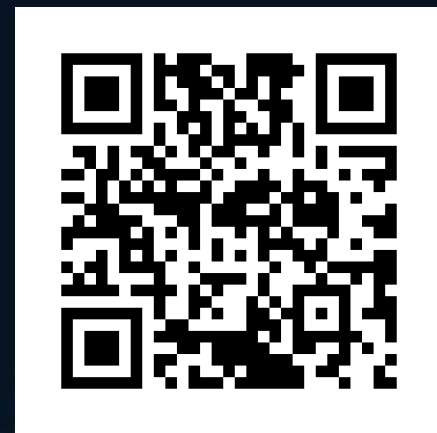
AI Infra 快速变革，持续追踪前沿
技术交流 · 经验分享 · 共同探索

跃跃欲试？

来做 AI Infra 工程实践！

HelloHPC 2nd · 校内赛

AI Infra 赛题，由浅入深
从轻松上手到进阶挑战



扫码报名，循序渐进练实战

AI Infra

微观

让一段计算更快

宏观

让整套系统更高效